KalSea: Kalman-Corrected Spectral-Evolution-Aware Caching

Anonymous CVPR submission



Figure 1. **Generated by our cache.** Each tile is a FLUX.1-dev image (1024×1024, 50 steps) generated with KalSea at $\delta = 0.2$, i.e. 27 of 50 transformer evaluations (1.85× fewer); the tiles are only downscaled, and the billboard is the central 2:1 band of its image. Prompts and seeds: supplement.

Abstract

Diffusion transformers spend most of their sampling time in the block stack, which runs at each of tens of sampling steps. Training-free caches skip it on some steps with a gate decides when *to compute, and a filler decides* what *to write on the skipped steps, typically the last block-stack residual or a forecast of it. Both fillers are open loop, when the next computed step reveals how wrong the filler was, its error has already been integrated into the latent and is never corrected. We close this loop with predictor-corrector gap smoothing (PCGS). Behind the spectrally aligned gate of SeaCache, a damped-trend Kalman filter forecasts the block-stack residual on skipped steps. When the next step is computed, a Rauch-Tung-Striebel smoother turns the filter's innovation into corrected residuals for the whole skipped gap, which we re-integrate through the output layers only before sampling continues from the corrected latent. PCGS requires no additional transformer evaluations. On FLUX.1-dev and held-out DrawBench prompts, our cache reaches a PSNR of 27.72 dB with respect to the 50-step sampler at 18 of 50 transformer evaluations, against 26.29 dB reported for SeaCache at 18, and 22.53 against 21.33 dB at 13 evaluations. In paired comparisons, PCGS alone contributes 0.81 dB (95% CI [0.63, 1.00]) and 0.61 dB [0.48, 0.76], and it also improves plain reuse and linear extrapolation. On HunyuanVideo and Wan2.1 at SeaCache's budgets, it improves PSNR over SeaCache run in the same setup by 1.9–3.2 dB and 0.3–2.3 dB.*

1. Introduction

Diffusion and flow-matching transformers [7, 33] generate high-resolution images and videos [3, 17, 47], but sampling is slow, each of tens of steps runs dozens of transformer blocks over thousands of tokens. Training-free caching exploits the similarity of consecutive steps [21, 30, 42], on some steps the block stack is skipped and its output is approximated from the computed steps. A cache thus makes two decisions: *when* to compute, and *what* to write on the steps it skips.

Most work has focused on the first decision. Adaptive gates accumulate a cheap proxy of the change of the network output and recompute once it exceeds a threshold [4, 21, 31].

SeaCache [6] measures this change after a spectral-evolution-aware (SEA) filter that suppresses the noise component of the features. For the second decision, most caches reuse the last computed output [6, 21, 30, 42], while forecasting methods extrapolate it from several computed steps with Taylor expansions [22], Hermite or Chebyshev polynomials [8, 10], or multistep formulas [54].

Reuse and forecasting share a structural limitation: they are *open loop*. The filler predicts the skipped steps from the past alone, and the sampler integrates its errors into the latent. These errors grow with the distance to the last computed step: on six reference trajectories of our development set, the relative error of the velocity obtained from a Kalman forecast of the residual grows from 4–5% at the first skipped step to over 20% at the seventh, as reported in the Sec. D of the supplementary material. When the next step is computed, its block-stack output reveals how wrong the forecast was, but existing caches use this observation only to predict the *next* gap; the latent, already advanced along erroneous velocities, is never revisited. Forecasting reduces this drift but does not remove it: as gaps grow, its advantage over reuse vanishes (Sec. 4.2).

We close the loop without evaluating the network again. Our key observation is that a skipped gap is a *smoothing* problem. We predict the block-stack residual with a Kalman filter. At the next computed step b, its innovation is the observed minus the forecast residual. The Rauch-Tung-Striebel (RTS) smoother [36] then uses this innovation and the covariances accumulated over the gap to compute the minimum-mean-square-error residuals of all the skipped steps under the filter's model. Because only the cheap output layers depend on the residual, we re-integrate the gap through them and continue sampling from the corrected latent. We call this procedure *predictor-corrector gap smoothing* (PCGS, Fig. 2). Like the corrector of UniPC [52], it reuses a network evaluation that the sampler performs anyway; unlike it, it corrects an entire gap of steps whose network outputs were never computed.

Combined with the unchanged SEA gate of SeaCache and a damped-trend Kalman forecaster of the block-stack residual, PCGS yields a training-free cache that changes neither the network, nor the sampler, nor the number of transformer evaluations. Our contributions are:

- We identify the open-loop nature of caching fillers: errors made on skipped steps are integrated into the latent and never corrected, and forecasting loses its advantage over reuse as gaps grow.
- We propose PCGS, which casts each skipped gap as a smoothing problem. Once the next step is computed, the RTS smoother of the forecaster's own state-space model gives the residuals of the gap, which is re-integrated with the output layers only. We give the smoother gain in closed form for the damped-trend model under the irregular observations of an adaptive gate. PCGS adds neither transformer evaluations nor hyperparameters.
- On FLUX.1-dev and held-out DrawBench prompts [37], our cache reaches 27.72 dB PSNR with respect to the 50-step sampler at 18 of 50 evaluations, against 26.29 dB published for SeaCache at 18, and 22.53 against 21.33 dB at 13. Against our re-run of SeaCache in the same setup, the paired gains are 2.39 dB [2.03, 2.75] and 1.27 dB [1.02, 1.52], of which PCGS contributes 0.81 and 0.61 dB. PCGS improves on the predictor at every budget from 9 to 27 evaluations and also improves reuse and linear extrapolation; its gain comes from re-integrating the gap rather than from the exact form of the smoother gains. On HunyuanVideo it improves on SeaCache in the same setup by 3.17 and 1.92 dB at 24 and 16 evaluations, and on Wan2.1 by 2.27 and 0.33 dB at 24 and 17 evaluations.

2. Related Work

Accelerating diffusion sampling. Sampling can be accelerated by taking fewer steps, by distilling the model, or by making each evaluation cheaper. Fast ODE solvers reduce the number of steps with higher-order or multistep integrators [16, 25, 26, 44, 52]. Distillation trains a few-step student, *e.g.*, by progressive distillation [38], consistency models [27, 46] or adversarial distillation [40]. Post-training quantization [18] lowers the precision of weights and activations. These methods change the sampler or the weights; caching leaves both unchanged and is orthogonal to them.

Training-free caching. Caching exploits the similarity of activations at consecutive sampling steps. Early caches reuse U-Net features [30, 50] or the blocks, attention and MLP outputs, and tokens of diffusion transformers [42, 55]; others target video [28, 53], distribute reuse over the trajectory [5] or learn which layers to cache [29]. Adaptive caches decide online when to recompute: AdaCache [14] adapts the schedule to each video, TeaCache [21] accumulates the relative change of the first block's timestep-modulated input until it exceeds a threshold, MagCache [31] models the magnitude ratio of successive residuals, DiCache [4] probes a shallow layer, and SADA [13] adapts the skipping to the stability of the sampling trajectory. SeaCache [6] measures this change after a spectral-evolution-aware filter, so that the gate responds to content rather than noise. Forecasting caches extrapolate the skipped outputs with Taylor expansions [22], Hermite [8] or Chebyshev [10] polynomials, or a BDF2 predictor with Heun-type calibration [54], and other methods verify [23] or rectify [34, 43] the value written on skipped steps. Concurrent work [35] also forecasts with Kalman filters. All of them improve the value used at the *current or upcoming* steps; none revises the latent of steps that have already been skipped.

Filtering, smoothing and predictor-corrector integration. The Kalman filter [15] and the RTS smoother [36] are the exact recursive estimators of linear-Gaussian state-space models [39]. Probabilistic ODE solvers infer the solution of an ODE from evaluations of its vector field by Gauss–Markov filtering and smoothing [41, 48]. In diffusion sampling, predictor-corrector schemes add Langevin steps [45] or a second evaluation as in Heun's method [16]; UniPC [52] corrects multistep solvers [26] by reusing the next evaluation.

3. Method

We keep the SEA gate of SeaCache and change what is written on skipped steps (Fig. 2). We review flow-matching sampling, residual caching and the gate (Sec. 3.1), then describe the Kalman predictor (Sec. 3.2), the corrector PCGS (Sec. 3.3) and the cost (Sec. 3.4).

3.1. Preliminaries

Flow-matching sampling. Flow matching [20], rectified flow [24] and stochastic interpolants [1] learn a velocity field that transports Gaussian noise to data. With the linear path and time convention of Esser *et al*. [7], data $\mathbf{x}^\star$ and noise $\boldsymbol{\epsilon}$ are interpolated at noise level $\sigma \in [0,1]$, and a network $\mathbf{v}_\theta$ is trained to predict the velocity of this path:

$$\mathbf{x}_\sigma = (1-\sigma)\mathbf{x}^\star + \sigma\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \tag{1}$$

$$\min_\theta \ \mathbb{E}\left\|\mathbf{v}_\theta(\mathbf{x}_\sigma, \sigma) - (\boldsymbol{\epsilon} - \mathbf{x}^\star)\right\|^2. \tag{2}$$

Sampling integrates the resulting ODE from $\sigma = 1$ (noise) to $\sigma = 0$ (data) over a decreasing schedule $\sigma_0 > \cdots > \sigma_N = 0$, *e.g*., with Euler steps:

$$\mathrm{d}\mathbf{x}_\sigma/\mathrm{d}\sigma = \mathbf{v}_\theta(\mathbf{x}_\sigma, \sigma), \tag{3}$$

$$\mathbf{x}_{i+1} = \mathbf{x}_i + (\sigma_{i+1} - \sigma_i)\,\mathbf{v}_i, \qquad \mathbf{v}_i = \mathbf{v}_\theta(\mathbf{x}_i, \sigma_i). \tag{4}$$

Residual caching. A diffusion transformer computes the velocity in three stages,

$$\mathbf{h}_i^{\mathrm{in}} = \mathrm{Emb}(\mathbf{x}_i), \quad \mathbf{h}_i^{\mathrm{out}} = \mathcal{F}(\mathbf{h}_i^{\mathrm{in}}), \quad \mathbf{v}_i = \mathrm{Out}(\mathbf{h}_i^{\mathrm{out}}, \mathbf{c}_i), \tag{5}$$

where Emb embeds the latent patches, $\mathcal{F}$ is the block stack (in FLUX.1-dev, 19 double-stream and 38 single-stream blocks, which also process the text tokens), Out is the final adaptive normalization and linear projection, and $\mathbf{c}_i$ embeds the timestep and the guidance scale. Almost all computation is in $\mathcal{F}$. Residual caches [6, 21] store the block-stack residual of the image tokens, $\mathbf{r}_i = \mathbf{h}_i^{\mathrm{out}} - \mathbf{h}_i^{\mathrm{in}}$, on computed steps. On skipped steps they run only Emb and Out, with $\mathbf{h}_i^{\mathrm{out}} = \mathbf{h}_i^{\mathrm{in}} + \hat{\mathbf{r}}_i$, where the filler $\hat{\mathbf{r}}_i$ is the last computed residual (reuse) or a forecast.

The SEA gate. TeaCache [21] and SeaCache [6] decide whether to compute from the timestep-modulated input of the first block, $\mathbf{I}_i$, *i.e*., its adaptive layer normalization applied to $\mathbf{h}_i^{\mathrm{in}}$. SeaCache filters $\mathbf{I}_i$ on the token grid with the normalized gain of the optimal linear denoiser under a power-law spectrum, $G_i(f) \propto a_i S(f)/(a_i^2 S(f) + b_i^2)$ with $S(f) = |f|^{-\beta}$, $a_i = 1 - \sigma_i$ and $b_i = \sigma_i$, and accumulates the relative ℓ_1 change of the filtered input,

$$d_i = \frac{\|\,\mathrm{SEA}_i(\mathbf{I}_i) - \mathrm{SEA}_{i-1}(\mathbf{I}_{i-1})\|_1}{\|\,\mathrm{SEA}_{i-1}(\mathbf{I}_{i-1})\|_1}, \quad D_i = D_{i-1} + d_i. \tag{6}$$

Step i is computed if $D_i \geq \delta$, after which D_i is reset to zero, or if it is the first or the last step; otherwise it is skipped. The threshold δ sets the budget. We use the official gate unchanged ($\beta = 2$ for images and 3 for videos).

3.2. Kalman residual forecasting (predictor)

The residual evolves smoothly but nonlinearly along the trajectory, and the cache observes it only at the irregular steps chosen by the gate. We model every element of the residual as a damped trend observed in noise [2, 9]. The state $\mathbf{s}_i = [m_i, \nu_i]^\top$ holds a value and its rate of change with respect to a time coordinate s that increases monotonically along the schedule, with steps $\Delta s_i = s(i) - s(i-1)$. At every step, computed or skipped, the filter predicts

$$\mathbf{s}_i^- = \mathbf{A}_i\,\mathbf{s}_{i-1}^+, \qquad \mathbf{P}_i^- = \mathbf{A}_i\,\mathbf{P}_{i-1}^+\mathbf{A}_i^\top + \mathbf{Q}_i, \tag{7}$$

$$\mathbf{A}_i = \begin{bmatrix} 1 & \Delta s_i \\ 0 & \rho_i \end{bmatrix}, \qquad \mathbf{Q}_i = q\kappa \begin{bmatrix} \Delta s_i^3/3 & \Delta s_i^2/2 \\ \Delta s_i^2/2 & \Delta s_i \end{bmatrix},$$

with $\rho_i = e^{-\Delta s_i/\tau}$. Here $\mathbf{Q}_i$ is the process noise of an integrated Wiener process [2] with intensity $q\kappa$, and ρ_i damps the trend so that forecasts over long gaps do not overshoot [9]. On a computed step b the filter observes the residual element z_b with noise variance R and updates, with $\mathbf{h} = [1, 0]^\top$,

$$e_b = z_b - m_b^-, \quad S_b = [\mathbf{P}_b^-]_{00} + R, \quad \mathbf{k}_b = \mathbf{P}_b^-\mathbf{h}/S_b,$$

$$\mathbf{s}_b^+ = \mathbf{s}_b^- + \mathbf{k}_b\,e_b, \qquad \mathbf{P}_b^+ = (\mathbf{I} - \mathbf{k}_b\mathbf{h}^\top)\,\mathbf{P}_b^-. \tag{8}$$

On skipped steps the moments are not updated ($\mathbf{s}_i^+ = \mathbf{s}_i^-$, $\mathbf{P}_i^+ = \mathbf{P}_i^-$), and the cache writes the predicted value, $\hat{\mathbf{r}}_i = m_i^-$, into $\mathbf{h}_i^{\mathrm{out}} = \mathbf{h}_i^{\mathrm{in}} + \hat{\mathbf{r}}_i$; the output layers run as usual. The intensity κ is adapted online so that the mean squared innovation matches its predicted variance S_b [32] (supplement).

Parametrizations. The covariance $\mathbf{P}$ is shared by all elements (per-element variant) or by all coefficients of a frequency band (DCT variant), so the filter costs a few element-wise operations per step and its gains are scalars per band. In the *per-element* variant, s is a warped log signal-to-noise ratio of the schedule. In the *DCT* variant (as in [35]), our

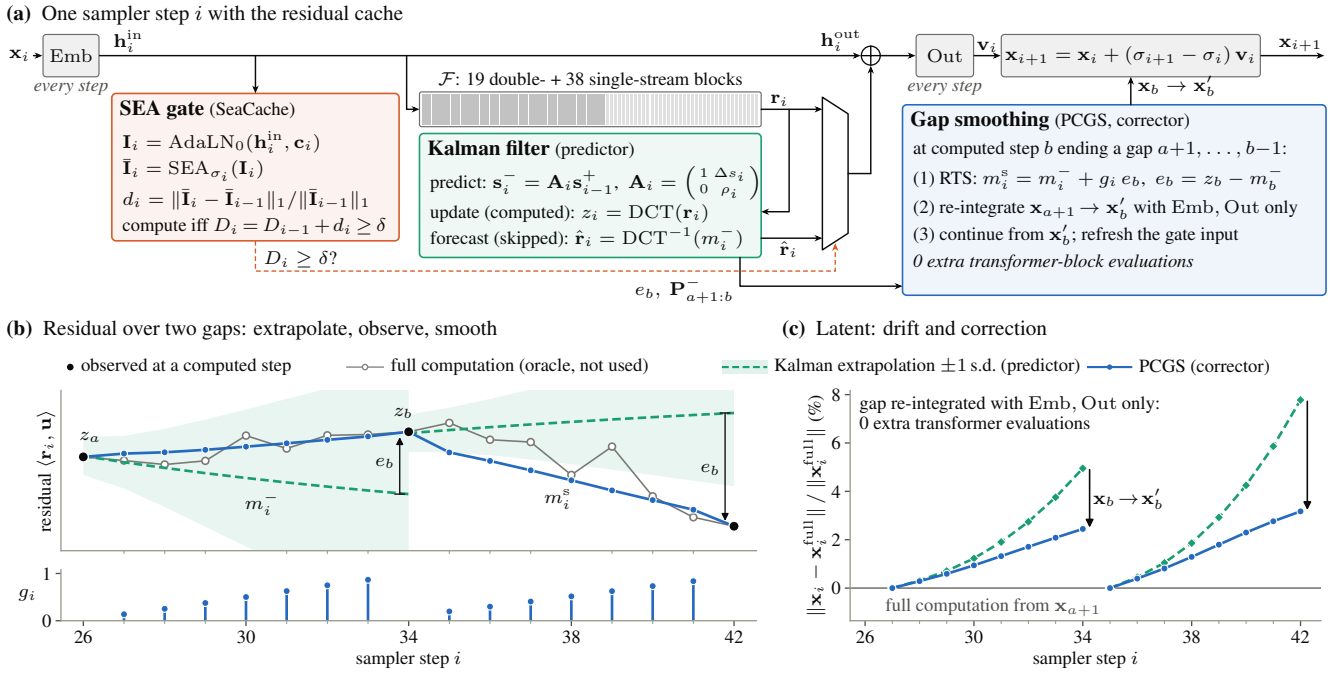


Figure 2. **Overview. (a)** In each step i, the SEA gate of SeaCache accumulates the SEA-filtered relative change d_i of the first block's modulated input and runs the blocks $\mathcal{F}$ only when the sum reaches δ; the residual $\mathbf{r}_i$ of a computed step updates a damped-trend Kalman filter per DCT coefficient (the predictor), whose forecast $\hat{\mathbf{r}}_i$ replaces it on skipped steps; Emb and Out run at every step. **(b)** When a computed step b ends the gap $a{+}1, \ldots, b{-}1$, PCGS spreads the innovation e_b over the gap with the RTS gains g_i, which grow toward b; the smoothed residuals m_i^{s} replace the extrapolation m_i^-. **(c)** Re-integrating the gap from $\mathbf{x}_{a+1}$ with Emb and Out only lowers the latent deviation from full computation at b by 25–59% in all ten gaps of this sample (here $5.0\% \to 2.4\%$ and $7.8\% \to 3.2\%$), without extra transformer evaluations. (b, c): one FLUX.1-dev generation at $\delta = 0.6$ for the DrawBench prompt whose PSNR gains (forecast over reuse, and gap smoothing over the forecast) are closest to the benchmark means ("McDonalds Church."); residual projected onto a fixed random unit direction $\mathbf{u}$ in the lowest DCT band (89% of its energy); gray: full computation, evaluated only for this figure.

default for FLUX.1-dev, the residual of each channel is transformed with an orthonormal 2D DCT on the token grid, the coefficients are grouped into seven radial frequency bands, and each band has its own warped time coordinate and covariance. Noise intensities, initial trend variances and band powers are calibrated once per model from 16 uncached trajectories of prompts disjoint from all evaluation prompts; $\tau = 0.5$ and $R = 10^{-3}$ throughout (supplement). The two variants perform the same (Sec. 4.4). With the corrector of Sec. 3.3, this predictor forms our full method, KalSea; the predictor alone (the SEA gate with Kalman residual forecasting) is labeled KalSea w/o PCGS in tables and figures.

3.3. Predictor-corrector gap smoothing (corrector)

Let $a < b$ be consecutive computed steps, so that $a{+}1, \ldots, b{-}1$ is a gap of skipped steps. The predictor has written $\hat{\mathbf{r}}_i = m_i^-$ on the gap, and the sampler has advanced the latent from $\mathbf{x}_{a+1}$ to $\mathbf{x}_b$ with the corresponding velocities. At step b the block stack runs and the filter observes z_b. The innovation e_b is the forecast error at the end of the gap; the forecast errors inside the gap are correlated with it, since they stem from the same trend error and process noise. PCGS exploits this correlation in four steps.

(1) Smoothing. No observation falls inside the gap, so $\mathbf{s}_b = \mathbf{\Phi}(b,i)\,\mathbf{s}_i + \mathbf{w}_{i\to b}$ for $a < i < b$, with the transition $\mathbf{\Phi}(b,i) = \mathbf{A}_b \mathbf{A}_{b-1} \cdots \mathbf{A}_{i+1}$ and accumulated process noise $\mathbf{w}_{i\to b}$ independent of $\mathbf{s}_i$. Given the observations up to step a, the cross-covariance of $\mathbf{s}_i$ and z_b is therefore $\mathbf{P}_i^- \mathbf{\Phi}(b,i)^\top \mathbf{h}$, and conditioning the joint Gaussian on z_b yields

$$m_i^{\mathrm{s}} = m_i^- + g_i\, e_b, \qquad g_i = \frac{\left[\mathbf{P}_i^- \mathbf{\Phi}(b,i)^\top\right]_{00}}{[\mathbf{P}_b^-]_{00} + R}. \tag{9}$$

This is the RTS smoother [36] over $[a,b]$, whose gain products telescope because the filtered and predicted moments coincide inside the gap (proof in the supplement). The gains use only quantities that the filter already maintains, so the smoother has no parameters of its own. They increase across the gap: the steps closest to b, whose forecasts were the least reliable, absorb most of the innovation. In residual space, the smoothed residual is the forecast plus a gain-weighted share of the forecast error revealed at b,

$$\tilde{\mathbf{r}}_i = \hat{\mathbf{r}}_i + \mathcal{G}_i\big(\mathbf{r}_b - \hat{\mathbf{r}}_b\big), \qquad a < i < b, \tag{10}$$

where $\hat{\mathbf{r}}_b = m_b^-$ is the forecast for step b, and $\mathcal{G}_i$ multiplies by the scalar g_i (per-element variant) or applies the band gains $g_{i,\ell}$ in the DCT domain.

(2) Re-integration. Only the output layers depend on the residual. We therefore replay the sampler over the gap with the smoothed residuals, starting from the latent that the sampler actually used at the first skipped step, $\mathbf{x}'_{a+1} = \mathbf{x}_{a+1}$; for $a < i < b$,

$$\mathbf{x}'_{i+1} = \mathbf{x}'_i + (\sigma_{i+1} - \sigma_i)\ \mathrm{Out}\left(\mathrm{Emb}(\mathbf{x}'_i) + \tilde{\mathbf{r}}_i, \mathbf{c}_i\right). \quad (11)$$

Re-embedding the corrected latents $\mathbf{x}'_i$, instead of reusing the stored inputs $\mathbf{h}_i^{\mathrm{in}}$, makes the replay an exact Euler integration with the smoothed residuals; a batched variant that reuses $\mathbf{h}_i^{\mathrm{in}}$ is ablated in Sec. 4.4.

(3) Continuation. The block stack at step b was evaluated on the uncorrected latent $\mathbf{x}_b$. Instead of evaluating it again, we keep its residual $\mathbf{r}_b$, attach it to the corrected input, and absorb the latent correction into the velocity returned to the sampler:

$$\mathbf{v}'_b = \mathrm{Out}\left(\mathrm{Emb}(\mathbf{x}'_b) + \mathbf{r}_b, \mathbf{c}_b\right) + \frac{\mathbf{x}'_b - \mathbf{x}_b}{\sigma_{b+1} - \sigma_b}. \quad (12)$$

The unmodified Euler step (4) then produces $\mathbf{x}_{b+1} = \mathbf{x}'_b + (\sigma_{b+1} - \sigma_b)\ \mathrm{Out}(\mathrm{Emb}(\mathbf{x}'_b) + \mathbf{r}_b, \mathbf{c}_b)$, *i.e.*, sampling continues from $\mathbf{x}'_b$, and PCGS remains a drop-in replacement of the network output.

(4) Gate refresh. The gate's reference input of step b is recomputed from $\mathbf{x}'_b$ (no measurable effect, Sec. 4.4).

Remarks. Under the filter's model and to first order, re-integrating with the smoothed residuals minimizes the expected squared error of $\mathbf{x}'_b$ given the observations up to step b (supplement). PCGS does not change the predictor for the next gap, because smoothed and filtered estimates coincide at the latest observation. It only requires the solver update to be linear in the network output: for a multistep solver such as the UniPC sampler [52] of Wan2.1, the solver state is restored at step $a{+}1$ and the updates are replayed with the corrected outputs; with classifier-free guidance [11], the conditional and unconditional residuals are filtered and smoothed separately under a shared gate. Algorithm 1 summarizes the procedure.

3.4. Cost

KalSea performs exactly the block-stack evaluations of the gate's schedule; PCGS adds none. It adds one evaluation of Emb and Out per skipped step and, at each computed step that closes a gap, one more of each plus the gate refresh:

Algorithm 1 KalSea sampling. The second **if** block is PCGS; without it, the procedure is the predictor alone (KalSea w/o PCGS).

Require: noise $\mathbf{x}_0$, schedule $\sigma_0 > \cdots > \sigma_N = 0$, threshold δ
$D \leftarrow 0$; gap $\mathcal{S} \leftarrow \emptyset$
for $i = 0, \ldots, N-1$ **do**
  $\mathbf{h}_i^{\mathrm{in}} \leftarrow \mathrm{Emb}(\mathbf{x}_i)$; $D \leftarrow D + d_i$; predict ▷ (6), (7)
  if $i \in \{0, N-1\}$ **or** $D \geq \delta$ **then** ▷ computed step, $b = i$
    $\mathbf{r}_i \leftarrow \mathcal{F}(\mathbf{h}_i^{\mathrm{in}}) - \mathbf{h}_i^{\mathrm{in}}$; $D \leftarrow 0$; Kalman update ▷ (8)
    $\mathbf{v}_i \leftarrow \mathrm{Out}(\mathbf{h}_i^{\mathrm{in}} + \mathbf{r}_i, \mathbf{c}_i)$
  else ▷ skipped step
    $\hat{\mathbf{r}}_i \leftarrow m_i^-$; $\mathbf{v}_i \leftarrow \mathrm{Out}(\mathbf{h}_i^{\mathrm{in}} + \hat{\mathbf{r}}_i, \mathbf{c}_i)$; $\mathcal{S} \leftarrow \mathcal{S} \cup \{i\}$
  if step i was computed **and** $\mathcal{S} \neq \emptyset$ **then**
    $\tilde{\mathbf{r}}_j \leftarrow \hat{\mathbf{r}}_j + \mathcal{G}_j(\mathbf{r}_i - \hat{\mathbf{r}}_i)$, $j \in \mathcal{S}$ ▷ (10)
    $\mathbf{x}' \leftarrow \mathbf{x}_{\min \mathcal{S}}$ ▷ (11)
    for $j \in \mathcal{S}$ in ascending order **do**
      $\mathbf{x}' \leftarrow \mathbf{x}' + (\sigma_{j+1} - \sigma_j)\ \mathrm{Out}(\mathrm{Emb}(\mathbf{x}') + \tilde{\mathbf{r}}_j, \mathbf{c}_j)$
    $\mathbf{v}_i \leftarrow \mathrm{Out}(\mathrm{Emb}(\mathbf{x}') + \mathbf{r}_i, \mathbf{c}_i) + \frac{\mathbf{x}' - \mathbf{x}_i}{\sigma_{i+1} - \sigma_i}$ ▷ (12)
    recompute the gate reference from $\mathbf{x}'$; $\mathcal{S} \leftarrow \emptyset$
  $\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + (\sigma_{i+1} - \sigma_i)\,\mathbf{v}_i$ ▷ unchanged sampler
return $\mathbf{x}_N$

for FLUX.1-dev at 1024^2, about 0.3% of one transformer evaluation per image. The filter state and the gap buffer raise the peak memory by 0.16 GiB over the uncached sampler (SeaCache: 0.05 GiB; KalSea w/o PCGS: 0.14 GiB; supplement).

4. Experiments

4.1. Setup

Models and protocol. We follow the protocol of SeaCache [6]. For text-to-image generation we use FLUX.1-dev [3] at $1024{\times}1024$ with 50 Euler steps of its default flow-matching schedule, embedded guidance 3.5, and the 200 DrawBench prompts [37]; the seed of each image is its prompt index. For text-to-video generation we use HunyuanVideo [17] at $480{\times}640$ and Wan2.1-T2V-1.3B [47] at $832{\times}480$, both with 65 frames, 50 steps and seed 42, on the 944 unique VBench prompts [12]. All methods share the seeds, and each accelerated sample is compared with the output of the uncached 50-step sampler for the same prompt and seed.

Budgets and baselines. We report the mean number of block-stack evaluations per image (NFE, of 50), counted by hooks on every block, and compare only at matched NFE. Every baseline runs in our setup with the same prompts and seeds: TeaCache, TaylorSeer, Spectrum [10], SADA [13] and fewer sampling steps with their released code, and SeaCache with the same gate as our caches. Our caches use the official SEA gate, whose threshold δ sets the budget. At $\delta = 0.6$ it computes 13.0 steps both in SeaCache's published

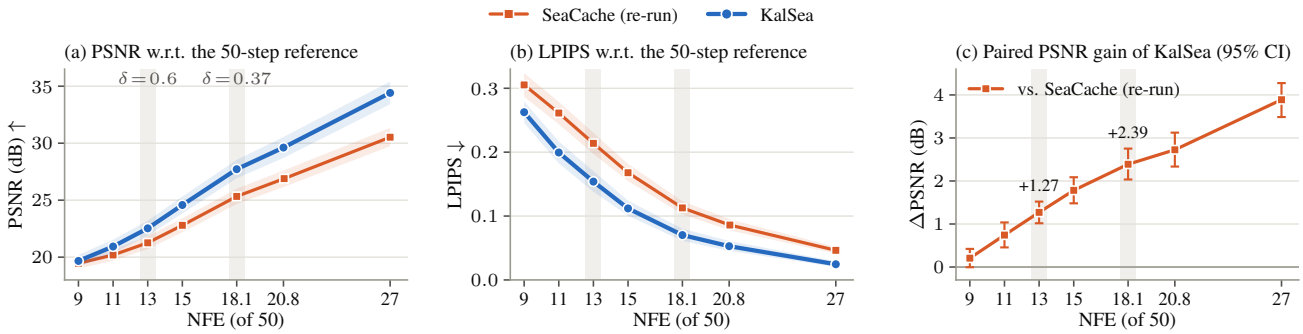


Figure 3. **Fidelity vs. compute on FLUX.1-dev** (DrawBench, 200 held-out prompts). **(a)** PSNR and **(b)** LPIPS w.r.t. the 50-step reference vs. the mean number of transformer evaluations per image; the SEA-gate threshold $\delta \in \{0.2, 0.3, 0.37, 0.5, 0.6, 0.8, 1.0\}$, shared by all methods, sets the budget, and gray bands mark the budgets of Tab. 1. All methods share code, seeds and hardware (SeaCache: our re-run of its released gate with reuse). Lines: means over prompts; shaded: 95% bootstrap CIs. **(c)** Paired PSNR gain of KalSea (95% CIs): over SeaCache, it grows from +0.21 dB [−0.005, +0.42] at 9 NFE (not significant) to +3.89 dB [+3.49, +4.28] at 27 NFE.

Table 1. **Text-to-image results on FLUX.1-dev** (DrawBench, 200 held-out prompts). Every row is run in our setup with the same prompts and seeds and compared with the uncached 50-step output of the same code: TeaCache, TaylorSeer (order 2, which scores higher than its default order 1), Spectrum, SADA (its default configuration and a faster one) and fewer sampling steps with their released code, SeaCache with the same gate as KalSea. One evaluation costs 59.5 TFLOPs in SeaCache's convention. Lat.: s/image on one B200, one prompt (supplement; –: not timed). IR: ImageReward; CLIP: CLIP score (ViT-L/14); both reference-free sanity checks (uncached 50-step output: 1.000 / 27.66).

Method	NFE	Lat.	PSNR↑	SSIM↑	LPIPS↓	IR↑	CLIP↑
Original (50 steps)	50	5.25	–	–	–	1.000	27.66
≈ 21 NFE (baselines: 21–26 NFE)							
TeaCache [21] (δ=0.3)	26.0	2.81	20.69	0.802	0.203	1.050	27.82
Fewer steps (25)	25.0	2.68	17.54	0.728	0.302	0.994	27.70
SADA [13] (default)	24.0	–	29.41	**0.935**	0.054	1.006	27.75
SADA [13] (faster)	21.5	–	25.67	0.892	0.099	1.000	27.71
SeaCache [6] (δ=0.3)	20.9	–	26.89	0.904	0.086	1.000	27.65
KalSea (δ=0.3)	20.8	–	**29.62**	**0.935**	**0.053**	1.013	27.70
18 NFE							
TaylorSeer [22] (S=3)	18.0	2.64	19.68	0.779	0.222	0.998	27.57
Spectrum [10] (α=0.3)	18.0	–	25.48	0.885	0.101	0.995	27.73
SeaCache [6] (δ=0.37)	18.1	2.05	25.33	0.880	0.113	0.996	27.71
KalSea (δ=0.37)	18.1	2.12	**27.72**	**0.916**	**0.070**	1.013	27.67
13 NFE (baselines: 12–15 NFE)							
TeaCache [21] (δ=0.6)	15.0	1.71	17.13	0.699	0.343	0.991	27.85
Fewer steps (15)	15.0	1.66	15.34	0.647	0.415	0.938	27.62
Spectrum [10] (α=1.0)	13.0	–	**23.72**	**0.847**	**0.142**	0.995	27.81
TaylorSeer [22] (S=5)	12.0	2.09	16.92	0.689	0.342	0.977	27.53
SeaCache [6] (δ=0.6)	13.0	1.53	21.26	0.800	0.214	0.973	27.80
KalSea (δ=0.6)	13.0	1.60	22.53	0.842	0.154	1.005	27.77

table and in our setup, where our re-run of SeaCache reproduces the published PSNR (21.26 vs. 21.33 dB); at $\delta = 0.3$ it computes 20.9 steps in our setup but 18.4 according to the published TFLOPs, so we match NFE rather than δ and run our caches at $\delta = 0.37$ (18.1 NFE) and 0.6. For controlled comparisons (same code, seeds and hardware), four fillers run behind the same gate: reuse (SeaCache), linear extrapolation of the residual from the last two computed steps (the first-order forecast of TaylorSeer [22]), the Kalman forecast (KalSea w/o PCGS), and KalSea. Behind the same gate, the caches compute the same steps on most prompts (supplement).

Metrics. PSNR on 8-bit RGB, SSIM [49] (scikit-image, 7×7 window) and LPIPS (AlexNet features [51]) are computed against the 50-step reference and averaged over prompts. Differences between methods run in the same setup come with paired bootstrap 95% confidence intervals over prompts (10,000 resamples, percentile) and the fraction of prompts on which a method is better.

Development and test prompts. All design decisions (the smoother, the re-integration and continuation rules, τ and R) were made on 64 COCO captions [19] and the forecaster was calibrated on 16 others, so DrawBench is a held-out test set (development-set results: supplement).

Implementation. For FLUX.1-dev, our cache replaces the transformer's forward pass in diffusers and leaves the scheduler unchanged. All runs use bf16 on NVIDIA B200 GPUs. For video we use the calibration-free per-element filter.

4.2. Comparison at matched NFE

Tab. 1 compares KalSea with all baselines in our setup. At 18.1 NFE, KalSea reaches 27.72 dB PSNR, 2.39 dB [2.03, 2.75] above SeaCache at the same NFE (better on 88% of the prompts) and 2.2 dB above Spectrum, with the best SSIM and LPIPS at this budget; it is also above the 26.29 dB

Table 2. **Text-to-video results** (944 VBench prompts, 65 frames, seed 42, fidelity to the 50-step output). Every row is run in our setup with the same prompts, seed and GPUs: fewer sampling steps, TeaCache and TaylorSeer with their released code, Spectrum in our implementation (identical to its released code on FLUX.1-dev), SeaCache with the same gate as KalSea. Thresholds and strides of all rows: supplement. VBench: Total score over all 16 dimensions (official weights and normalization; uncached in the first row). Lat.: s/video on one B200, one prompt (supplement; –: not timed).

	HunyuanVideo, 480×640, 65 frames						Wan2.1 1.3B, 832×480, 65 frames					
Method	NFE	Lat. (s)	PSNR↑	SSIM↑	LPIPS↓	VBench↑	NFE	Lat. (s)	PSNR↑	SSIM↑	LPIPS↓	VBench↑
Reference (50 steps)	50	74.5	–	–	–	81.02	50	52.0	–	–	–	80.56
≈ 50% budget (23–27 NFE)												
Fewer steps	25.0	39.9	20.14	0.702	0.254	80.76	–	–	–	–	–	–
TeaCache [21]	24.8	40.2	22.78	0.766	0.184	80.83	25.0	27.6	22.85	0.834	0.110	80.73
TaylorSeer [22]	25.0	43.3	20.81	0.719	0.230	81.29	25.0	29.1	14.77	0.571	0.378	80.38
Spectrum [10]	23.0	–	27.31	0.853	0.104	80.79	27.0	–	22.62	0.821	0.109	80.72
SeaCache [6]	24.0	38.4	32.43	0.929	0.048	–	24.0	26.1	29.40	0.931	0.043	–
KalSea	24.0	38.6	**35.60**	**0.951**	**0.028**	81.11	24.0	26.5	**31.67**	**0.953**	**0.020**	80.36
≈ 30% budget (15–19 NFE)												
Fewer steps	15.0	25.9	17.26	0.612	0.375	79.78	–	–	–	–	–	–
TeaCache [21]	17.0	28.9	20.03	0.687	0.262	80.41	18.0	20.6	20.45	0.768	0.172	79.73
TaylorSeer [22]	17.0	32.6	18.01	0.629	0.328	81.01	17.0	20.9	13.55	0.513	0.452	79.41
Spectrum [10]	16.0	–	26.22	0.818	0.126	80.48	19.0	–	21.04	0.744	0.140	79.53
SeaCache [6]	16.0	27.4	24.53	0.813	0.155	80.67	17.0	19.1	23.52	0.828	0.118	–
KalSea	16.0	27.5	**26.45**	**0.846**	**0.117**	81.19	17.0	19.4	**23.85**	**0.851**	**0.090**	80.74

that SeaCache reports at 18.4 NFE. At 20.8 NFE it reaches 29.62 dB and exceeds every baseline that uses more evaluations, including SADA's default configuration at 24.0 NFE (29.41 dB). At 13 NFE, it improves over SeaCache by 1.27 dB [1.02, 1.52] (79%; 22.53 vs. 21.26 dB, reported: 21.33 dB) and over TeaCache, TaylorSeer and fewer steps, but Spectrum is 1.2 dB higher; its fixed schedule always computes the first five steps, which the SEA gate partly skips at $\delta = 0.6$. ImageReward and CLIP score stay at the level of the uncached sampler, so the gain in fidelity does not come at a cost in perceived quality.

Budget sweep. Fig. 3 varies δ from 0.2 to 1.0 in the same setup. KalSea attains the highest PSNR and the lowest LPIPS at every budget; its PSNR gain over our re-run of SeaCache grows from 0.74 dB [0.46, 1.04] at 11 NFE to 3.89 dB [3.49, 4.28] at 27 NFE, and at 9 NFE only LPIPS improves significantly (0.043 [0.033, 0.053]).

Qualitative results. Fig. 4 shows one example per model at the lower budget, chosen by a fixed rule; SeaCache alters details that KalSea keeps, such as the street lamp added at the top right of the FLUX.1-dev image. The supplement shows more images and videos; Fig. 1 shows images generated by KalSea at 27 NFE.

4.3. Video generation

We apply our cache with the per-element filter to HunyuanVideo and Wan2.1-1.3B at SeaCache's two budgets (Tab. 2); on Wan2.1 (UniPC, classifier-free guidance) PCGS replays the solver (Sec. 3.3). SeaCache's released HunyuanVideo script gates on a stale noise level near one, which skips step 1 once $\delta \gtrsim 0.21$ and costs 12–14 dB on those prompts (supplement), so every gated method reads the sampler's noise level, with δ set for 24.0 and 16.0 NFE. Our re-run of SeaCache then reproduces its published result at 24 NFE (32.43 vs. 32.39 dB) but not at 16.4 (24.53 dB at 16.0 vs. 26.46 dB; its released gate gives 20.52 dB). All rows of Tab. 2 are run in our setup. On HunyuanVideo, KalSea improves on SeaCache by 3.17 dB [2.99, 3.35] at 24 NFE and by 1.92 dB [1.70, 2.14] at 16 NFE (better on 90% and 72% of the prompts), with better SSIM and LPIPS (examples: Fig. 4); on Wan2.1 the paired gains are 2.27 dB [2.13, 2.42] and 0.33 dB [0.16, 0.49], the latter with clearly better SSIM (+0.023) and LPIPS (−0.029). KalSea also leads every other baseline at both budgets, by 8.3 and 0.2 dB over the strongest on HunyuanVideo (Spectrum) and by 8.8 and 2.8 dB on Wan2.1 (TeaCache at 25 NFE, Spectrum at 19), although several of them use more evaluations. On VBench, every method stays within about one point of the uncached sampler; KalSea scores above it on HunyuanVideo (81.11 and 81.19 vs. 81.02) and at the smaller Wan2.1 budget (80.74 vs. 80.56), and 0.2 below it at the larger one (80.36), so the gains in fidelity come without a loss in VBench quality.

4.4. Ablations

Anatomy of the corrector. Tab. 3 reports paired PSNR differences to KalSea at $\delta = 0.3$ (20.8 NFE) and 0.6; differences below 0.1 dB are within run-to-run noise (supplement). Without PCGS, PSNR drops by 0.83 and 0.61 dB (Kalman

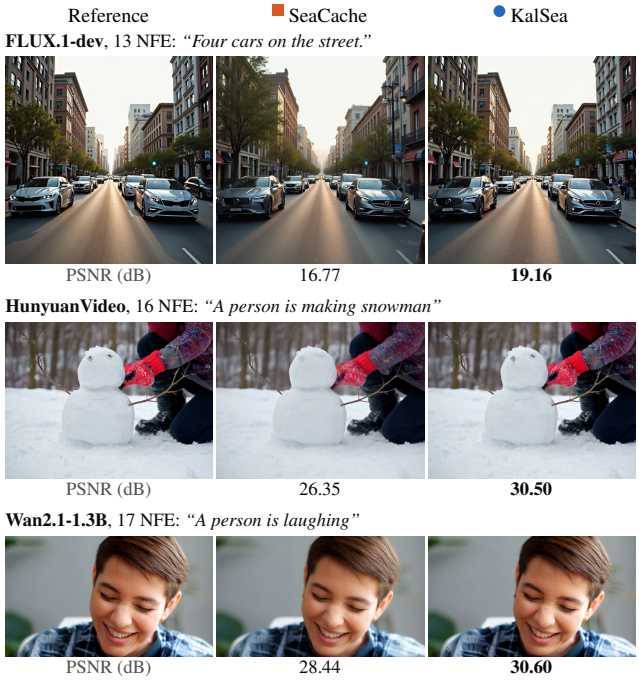


Figure 4. **One example per model at its lower budget**: FLUX.1-dev ($\delta = 0.6$, 13 NFE), HunyuanVideo ($\delta = 0.48$, 16 NFE, gate on the sampler's noise level), Wan2.1-1.3B ($\delta = 0.35$, 17 NFE); video: frame 32 of 65. Fixed rule: among the prompts on which SeaCache and KalSea computed identical steps, the one whose PSNR gain of KalSea over SeaCache is closest to the upper quartile. Numbers: per-image and per-video PSNR (dB) w.r.t. the 50-step reference. More in the supplement.

row); across budgets, the predictor alone loses its advantage over reuse as gaps grow, while PCGS improves on it at every budget (supplement). The gain comes from re-integrating the latent: feeding the smoothed residuals only into the filter state is no better than no correction (-0.86 vs. -0.83 dB at 20.8 NFE). Simpler gain shapes, such as a linear interpolation from $\mathbf{r}_a$ to $\mathbf{r}_b$, do as well as the RTS gains (supplement). The batched re-integration on the stored inputs $\mathbf{h}_i^{\text{in}}$ loses 0.22 dB [0.09, 0.34] at 20.8 NFE, and the gate refresh has no measurable effect.

Other forecasters and fillers. With PCGS, the forecaster's parametrization hardly matters: the per-element filter, a shorter damping $\tau = 0.1$ and a calibration-free filter all stay within 0.1 dB of KalSea, so the frequency decomposition is unnecessary for the residual and we use the per-element filter for video. Nor is PCGS tied to a Kalman forecaster: it improves reuse by 1.69 and 0.74 dB and linear extrapolation by 1.15 and 0.80 dB (supplement); linear extrapolation with PCGS then ties KalSea, while reuse with PCGS stays 1.03 and 0.53 dB below.

Table 3. **Ablations** (FLUX.1-dev, DrawBench-200, held out). First row: PSNR of KalSea; other rows: paired PSNR difference to KalSea (dB, 95% bootstrap CI). All rows share the SEA gate and threshold, with NFE within 0.06 (0.00) of KalSea at δ=0.3 (0.6). All metrics: supplement.

Variant	20.8 NFE (δ=0.3)	13 NFE (δ=0.6)
KalSea (Kalman + PCGS)	29.62 dB	22.53 dB
(a) Filler on skipped steps, no correction		
Reuse (SeaCache)	−2.73 [−3.12, −2.34]	−1.27 [−1.52, −1.02]
Linear extrapolation	−1.11 [−1.34, −0.89]	−0.73 [−0.92, −0.54]
Kalman (KalSea w/o PCGS)	−0.83 [−1.02, −0.63]	−0.61 [−0.76, −0.48]
(b) Correction of the Kalman forecast		
Filter state only	−0.86 [−1.06, −0.68]	−0.63 [−0.78, −0.50]
Linear interpolation	+0.14 [+0.04, +0.25]	+0.00 [−0.05, +0.06]
Forecast + lin. innov.	+0.14 [+0.04, +0.25]	+0.02 [−0.03, +0.07]
Batched re-integration	−0.22 [−0.34, −0.09]	−0.06 [−0.13, +0.01]
No gate refresh	+0.03 [−0.04, +0.10]	+0.01 [−0.01, +0.03]
(c) Other forecaster, with PCGS		
Per-element filter	+0.01 [−0.15, +0.14]	−0.01 [−0.15, +0.13]
Damping τ=0.1	−0.08 [−0.29, +0.12]	+0.05 [−0.11, +0.22]
Calibration-free	−0.02 [−0.11, +0.08]	+0.02 [−0.13, +0.17]
(d) PCGS on other fillers		
Reuse + PCGS	−1.03 [−1.39, −0.67]	−0.53 [−0.76, −0.29]
Linear extrap. + PCGS	+0.04 [−0.08, +0.16]	+0.07 [−0.05, +0.18]

5. Conclusion

Caching fillers for diffusion transformers are open loop. PCGS closes this loop at no cost in network evaluations: once the next step is computed, it estimates the residuals of the skipped gap with the RTS smoother of the forecaster's own state-space model and re-integrates the gap through the output layers. With the SEA gate and a Kalman residual forecaster, it improves fidelity to the 50-step sampler at every budget we tested on FLUX.1-dev, and the gains carry over to HunyuanVideo and Wan2.1. The benefit lies in the re-integration itself: a linear interpolation across the gap works as well as the RTS gains, and the correction also repairs reuse and linear extrapolation. Promising directions are to choose the computed steps jointly with the corrector and to correct caches that operate on features inside the block stack.

Limitations. Matched NFE is not matched latency: PCGS adds output-layer evaluations (Sec. 3.4) and 0.02 GiB over KalSea w/o PCGS, and the filter adds elementwise work: at equal NFE, KalSea is 3–5% slower than SeaCache on FLUX.1-dev and at most 1.4% on video. Our metrics measure fidelity to the 50-step sampler, the goal of a cache, not sample quality. Our image results cover a single model. On Wan2.1, the same-setup PSNR gain is only 0.33 dB at 17 NFE and negative at 10 NFE on 20 prompts (supplement). PCGS attaches the residual of the uncorrected latent to the corrected one, accurate only for small corrections; with gaps of up to twelve steps its gain is modest (0.31 dB at 9 NFE).

LLM Usage Statement

LLM-based assistants were used in this work to propose and implement method variants and experiment code, to run and monitor experiments and evaluation scripts, to draft and edit text and figures, and to check the reported numbers against the logged results. The authors directed the research and take full responsibility for the content of this paper.

References

[1] Michael S. Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations (ICLR)*, 2023. 3

[2] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001. 3

[3] Black Forest Labs. FLUX, 2024. Open-source code and model weights. 1, 5

[4] Jiazi Bu, Pengyang Ling, Yujie Zhou, Yibin Wang, Yuhang Zang, Dahua Lin, and Jiaqi Wang. DiCache: Let diffusion model determine its own cache. In *International Conference on Learning Representations (ICLR)*, 2026. 1, 2

[5] Huanpeng Chu, Wei Wu, Guanyu Feng, and Yutao Zhang. OmniCache: A trajectory-oriented global perspective on training-free cache reuse for diffusion transformer models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 16302–16312, 2025. 2

[6] Jiwoo Chung, Sangeek Hyun, MinKyu Lee, Byeongju Han, Geonho Cha, Dongyoon Wee, Youngjun Hong, and Jae-Pil Heo. SeaCache: Spectral-evolution-aware cache for accelerating diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14283–14294, 2026. 2, 3, 5, 6, 7

[7] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, pages 12606–12633, 2024. 1, 3

[8] Liang Feng, Shikang Zheng, Jiacheng Liu, Yuqi Lin, Qinming Zhou, Peiliang Cai, Xinyu Wang, Junjie Chen, Chang Zou, Yue Ma, and Linfeng Zhang. HiCache: A plug-in scaled-hermite upgrade for taylor-style cache-then-forecast diffusion acceleration. In *International Conference on Learning Representations (ICLR)*, 2026. 2

[9] Everette S. Gardner and Ed McKenzie. Forecasting trends in time series. *Management Science*, 31(10):1237–1246, 1985. 3

[10] Jiaqi Han, Juntong Shi, Puheng Li, Haotian Ye, Qiushan Guo, and Stefano Ermon. Adaptive spectral feature forecasting for diffusion sampling acceleration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 43320–43330, 2026. 2, 5, 6, 7

[11] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. 5

[12] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21807–21818, 2024. 5

[13] Ting Jiang, Yixiao Wang, Hancheng Ye, Zishan Shao, Jingwei Sun, Jingyang Zhang, Zekai Chen, Jianyi Zhang, Yiran Chen, and Hai Li. SADA: Stability-guided adaptive diffusion acceleration. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. 2, 5, 6

[14] Kumara Kahatapitiya, Haozhe Liu, Sen He, Ding Liu, Menglin Jia, Chenyang Zhang, Michael S. Ryoo, and Tian Xie. Adaptive caching for faster video generation with diffusion transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15240–15252, 2025. 2

[15] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960. 3

[16] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 26565–26577, 2022. 2, 3

[17] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, Kathrina Wu, Qin Lin, et al. HunyuanVideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024. 1, 5

[18] Xiuyu Li, Yijiang Liu, Long Lian, Huanrui Yang, Zhen Dong, Daniel Kang, Shanghang Zhang, and Kurt Keutzer. Q-Diffusion: Quantizing diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 17535–17545, 2023. 2

[19] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014. 6

[20] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. 3

[21] Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It's time to cache for video diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7353–7363, 2025. 1, 2, 3, 6, 7

[22] Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Junjie Chen, and Linfeng Zhang. From reusing to forecasting: Accelerating diffusion models with TaylorSeers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15853–15863, 2025. 2, 6, 7

[23] Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Fei Ren, Shaobo Wang, Kaixin Li, and Linfeng Zhang. SpeCa: Accelerating

diffusion transformers with speculative feature caching. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 10024–10033, 2025. 2

[24] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023. 3

[25] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5775–5787, 2022. 2

[26] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver++: Fast solver for guided sampling of diffusion probabilistic models. *Machine Intelligence Research*, 22(4):730–751, 2025. 2, 3

[27] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. *arXiv preprint arXiv:2310.04378*, 2023. 2

[28] Zhengyao Lv, Chenyang Si, Junhao Song, Zhenyu Yang, Yu Qiao, Ziwei Liu, and Kwan-Yee K. Wong. FasterCache: Training-free video diffusion model acceleration with high quality. In *International Conference on Learning Representations (ICLR)*, 2025. 2

[29] Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 133282–133304, 2024. 2

[30] Xinyin Ma, Gongfan Fang, and Xinchao Wang. DeepCache: Accelerating diffusion models for free. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15762–15772, 2024. 1, 2

[31] Zehong Ma, Longhui Wei, Feng Wang, Shiliang Zhang, and Qi Tian. MagCache: Fast video generation with magnitude-aware cache. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 38592–38624, 2025. 1, 2

[32] R. Mehra. Approaches to adaptive filtering. *IEEE Transactions on Automatic Control*, 17(5):693–698, 1972. 3

[33] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4195–4205, 2023. 1

[34] Xurui Peng, Chenqian Yan, Hong Liu, Rui Ma, Fangmin Chen, Xing Wang, Zhihua Wu, Songwei Liu, and Mingbao Lin. ERTACache: Error rectification and timesteps adjustment for efficient diffusion. In *International Conference on Learning Representations (ICLR)*, 2026. 2

[35] Javad Rajabi, Hakki Can Karaimer, David B. Lindell, Babak Taati, and Michael S. Brown. SpecKCast: Spectral Kalman forecasting for diffusion acceleration. *arXiv preprint arXiv:2610.26837*, 2026. 2, 3

[36] H. E. Rauch, F. Tung, and C. T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8): 1445–1450, 1965. 2, 3, 4

[37] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L. Denton, Seyed Kamyar Seyed Ghasemipour, Raphael Gontijo Lopes, Burcu Karagol Ayan, Tim Salimans, Jonathan Ho, David J. Fleet, and Mohammad Norouzi. Photorealistic text-to-image diffusion models with deep language understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 36479–36494, 2022. 2, 5

[38] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations (ICLR)*, 2022. 2

[39] Simo Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013. 3

[40] Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial diffusion distillation. In *European Conference on Computer Vision (ECCV)*, pages 87–103, 2024. 2

[41] Michael Schober, Simo Särkkä, and Philipp Hennig. A probabilistic model for the numerical solution of initial value problems. *Statistics and Computing*, 29(1):99–122, 2019. 3

[42] Pratheba Selvaraju, Tianyu Ding, Tianyi Chen, Ilya Zharkov, and Luming Liang. FORA: Fast-forward caching in diffusion transformer acceleration. *arXiv preprint arXiv:2407.01425*, 2024. 1, 2

[43] Zhirong Shen, Rui Huang, Chang Zou, Shikang Zheng, Jiacheng Liu, Peiliang Cai, Zhengyi Shi, Yaosong Du, Liang Feng, Xiaobing Tu, Jinkui Ren, Xiantao Zhang, and Linfeng Zhang. Accelerating diffusion transformers with Gaussian process rectified feature cache. In *European Conference on Computer Vision (ECCV)*, 2026. 2

[44] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021. 2

[45] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. 3

[46] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 32211–32252, 2023. 2

[47] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. 1, 5

[48] Filip Tronarp, Hans Kersting, Simo Särkkä, and Philipp Hennig. Probabilistic solutions to ordinary differential equations as nonlinear Bayesian filtering: A new perspective. *Statistics and Computing*, 29(6):1297–1315, 2019. 3

[49] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. 6

[50] Felix Wimbauer, Bichen Wu, Edgar Schoenfeld, Xiaoliang Dai, Ji Hou, Zijian He, Artsiom Sanakoyeu, Peizhao Zhang, Sam Tsai, Jonas Kohler, Christian Rupprecht, Daniel Cremers, Peter Vajda, and Jialiang Wang. Cache me if you can: Accelerating diffusion models through block caching. In *Pro-*

ceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 6211–6220, 2024. 2

[51] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 586–595, 2018. 6

[52] Wenliang Zhao, Lujia Bai, Yongming Rao, Jie Zhou, and Jiwen Lu. UniPC: A unified predictor-corrector framework for fast sampling of diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 49842–49869, 2023. 2, 3, 5

[53] Xuanlei Zhao, Xiaolong Jin, Kai Wang, and Yang You. Real-time video generation with pyramid attention broadcast. In *International Conference on Learning Representations (ICLR)*, 2025. 2

[54] Shikang Zheng, Liang Feng, Xinyu Wang, Qinming Zhou, Peiliang Cai, Chang Zou, Jiacheng Liu, Yuqi Lin, Junjie Chen, Yue Ma, and Linfeng Zhang. Forecast then calibrate: Feature caching as ODE for efficient diffusion transformers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 13449–13457, 2026. 2

[55] Chang Zou, Xuyang Liu, Ting Liu, Siteng Huang, and Linfeng Zhang. Accelerating diffusion transformers with token-wise feature caching. In *International Conference on Learning Representations (ICLR)*, 2025. 2